LAPORAN

WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)

SEMESTER III



BKPM Acara 20 API Authentication dan Error Handling

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

RESTful API merupakan layanan yang memungkinkan aplikasi saling bertukar data melalui protokol HTTP. Dalam pengembangan aplikasi web, API digunakan untuk menjalankan berbagai operasi, seperti mengambil data, menambahkan data, memperbarui data, dan menghapus data.

Salah satu aspek penting dalam pengembangan RESTful API adalah autentikasi. Autentikasi digunakan untuk memverifikasi identitas pengguna sebelum memberikan akses terhadap layanan tertentu. Tanpa autentikasi, endpoint yang mengelola data dapat diakses oleh pihak yang tidak memiliki izin.

JSON Web Token (JWT) merupakan salah satu mekanisme autentikasi yang dapat digunakan pada RESTful API. Setelah pengguna berhasil melakukan login, server menghasilkan token yang kemudian dikirimkan oleh client pada request berikutnya. Server memverifikasi token tersebut sebelum memproses permintaan ke endpoint yang dilindungi.

Selain autentikasi, aplikasi backend juga memerlukan mekanisme error handling. Penanganan kesalahan dilakukan agar aplikasi dapat memberikan respons yang sesuai ketika terjadi kesalahan, seperti data tidak valid, token tidak ditemukan, atau kesalahan pada proses basis data.

Berdasarkan hal tersebut, praktikum Acara 20 membahas implementasi autentikasi API menggunakan JWT serta penanganan kesalahan pada aplikasi berbasis PHP. Implementasi ini melanjutkan fitur registrasi pengguna yang telah dikembangkan pada Acara 19 dan menggunakan Postman untuk pengujian endpoint.

1.2 Rumusan Masalah

1. Bagaimana mengimplementasikan autentikasi API menggunakan JSON Web Token (JWT)?
2. Bagaimana menerapkan proteksi endpoint dan error handling pada aplikasi RESTful API berbasis PHP?

1.3 Tujuan Praktikum

1. Memahami konsep autentikasi menggunakan JWT pada RESTful API.
2. Mengimplementasikan proses login dan pembuatan token autentikasi.

3. Melindungi endpoint tertentu menggunakan validasi token.
4. Menerapkan mekanisme penanganan kesalahan menggunakan `try-catch`.
5. Menguji alur autentikasi dan respons kesalahan API menggunakan Postman.

BAB 2. LANDASAN TEORI

2.1 RESTful API

RESTful API merupakan antarmuka yang memungkinkan pertukaran data melalui HTTP dengan menggunakan metode seperti GET, POST, PUT, PATCH, dan DELETE. Setiap metode memiliki fungsi yang berbeda, misalnya GET untuk mengambil data dan POST untuk mengirimkan data baru.

RESTful API umumnya mengirimkan dan menerima data dalam format JSON sehingga mudah digunakan oleh berbagai aplikasi client.

2.2 Authentication

Authentication atau autentikasi merupakan proses verifikasi identitas pengguna atau aplikasi sebelum diberikan akses terhadap suatu layanan. Pada API, autentikasi dapat dilakukan menggunakan API Key, Basic Authentication, maupun Bearer Token.

Dalam praktikum ini, autentikasi dilakukan melalui proses login. Pengguna mengirimkan username dan password, kemudian server memverifikasi kredensial tersebut. Apabila valid, server menghasilkan token yang dapat digunakan untuk mengakses endpoint yang dilindungi.

2.3 JSON Web Token (JWT)

JSON Web Token adalah format token ringkas yang digunakan untuk menyampaikan informasi atau klaim antara pihak-pihak yang berkomunikasi. JWT umumnya terdiri atas tiga bagian, yaitu:

1. **Header**, berisi informasi mengenai tipe token dan algoritma tanda tangan.
2. **Payload**, berisi klaim, seperti identitas pengguna, peran, dan waktu kedaluwarsa token.
3. **Signature**, digunakan untuk memverifikasi keaslian token dan memastikan bahwa isinya tidak diubah tanpa terdeteksi.

Ketiga bagian tersebut dipisahkan oleh tanda titik. Pada praktikum ini, pembuatan tanda tangan token menggunakan algoritma HMAC-SHA256 melalui fungsi `hash_hmac()` pada PHP.

Token yang dikirimkan oleh client menggunakan header `Authorization` dengan skema Bearer Token. Server kemudian memverifikasi tanda tangan dan waktu kedaluwarsa token sebelum memberikan akses.

2.4 Middleware Autentikasi

Middleware autentikasi merupakan lapisan pemeriksaan yang dijalankan sebelum proses utama suatu endpoint. Middleware membaca token dari header request dan memvalidasi token tersebut.

Jika token tidak ditemukan atau tidak valid, server mengembalikan respons kesalahan dan menghentikan proses. Jika token valid, permintaan dapat dilanjutkan sesuai aturan akses yang diterapkan.

2.5 Error Handling

Error handling merupakan mekanisme untuk mendeteksi dan menangani kesalahan selama program berjalan. Pada PHP, salah satu mekanisme yang digunakan adalah `try-catch`.

Blok `try` digunakan untuk menjalankan proses yang berpotensi menimbulkan kesalahan, sedangkan blok `catch` menangani kesalahan yang dilemparkan oleh program atau pustaka yang digunakan.

Pada RESTful API, kesalahan perlu disampaikan melalui status HTTP dan respons JSON yang konsisten. Beberapa status yang umum digunakan adalah:

Kode	Keterangan
400 Bad Request:	permintaan tidak memenuhi ketentuan.
401 Unauthorized:	autentikasi tidak tersedia atau tidak valid.
403 Forbidden:	pengguna tidak memiliki izin untuk melakukan tindakan tertentu.
404 Not Found:	endpoint atau data tidak ditemukan.
409 Conflict:	terjadi konflik data.
500 Internal Server Error:	terjadi kesalahan internal pada server.

2.6 Postman

Postman merupakan aplikasi untuk mengirim request HTTP dan memeriksa respons API. Postman dapat digunakan untuk menguji proses login, melihat token

yang dihasilkan, menambahkan header Authorization, serta memeriksa status HTTP dan pesan kesalahan.

BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktikum

3.1.1 Studi Kasus

Studi kasus pada Acara 20 adalah pengembangan autentikasi API pada Sistem Informasi Akademik. Pengembangan dilakukan dengan melanjutkan fitur registrasi pengguna dari Acara 19.

Pengguna yang sudah terdaftar dapat melakukan login menggunakan username dan password. Apabila kredensial sesuai, server menghasilkan JWT. Token tersebut digunakan untuk mengakses operasi tertentu pada endpoint mahasiswa.

Pada praktikum ini, endpoint GET mahasiswa tetap dapat diakses tanpa token untuk keperluan pengujian, sedangkan operasi penambahan, perubahan, dan penghapusan data mahasiswa dilindungi oleh autentikasi.

A. Langkah 1 — Membuat Class Auth pada Lapisan Core `app/core/Auth.php`

Langkah pertama adalah membuat file `Auth.php` pada direktori `app/core/`. Class `Auth` digunakan untuk menghasilkan token, memvalidasi token, dan memeriksa autentikasi pengguna melalui header `Authorization`.

Kode utama yang digunakan untuk menghasilkan token adalah sebagai berikut:

```php
<?php
class Auth {
    const SECRET = 'kunci_rahasia_workshop_2026';
    // Tugas Mandiri 2: masa berlaku token 5 menit (300
detik)
    const TOKEN_TTL = 300;

    private static function base64url(string $data):
string {
        return rtrim(strtr(base64_encode($data), '+/', '-
_'), '=');
    }
```

```
    private static function base64urlDecode(string $data)
{
        $sisa = strlen($data) % 4;
        if ($sisa) {
            $data .= str_repeat('=', 4 - $sisa);
        }
        return base64_decode(strtr($data, '-_', '+/'),
true);
    }

    public static function generateToken(array $payload):
string {
        $header     = self::base64url(json_encode(['alg'
=> 'HS256', 'typ' => 'JWT']));
        $payload['exp'] = time() + self::TOKEN_TTL;
        $payloadEnc =
self::base64url(json_encode($payload));
        $signature  = self::base64url(hash_hmac('sha256',
"$header.$payloadEnc", self::SECRET, true));
        return "$header.$payloadEnc.$signature";
    }

    public static function validateToken(string $token) {
        $parts = explode('.', $token);
        if (count($parts) !== 3) return false;

        [$header, $payload, $signature] = $parts;

        $valid = self::base64url(hash_hmac('sha256',
"$header.$payload", self::SECRET, true));
        if (!hash_equals($valid, $signature)) return
false;

        $json = self::base64urlDecode($payload);
```

```
        $data = $json === false ? null :
json_decode($json, true);
        if (!is_array($data) || !isset($data['exp']) ||
$data['exp'] < time()) return false;

        return $data;
    }

    // Ambil header Authorization (tidak peka huruf
besar/kecil)
    private static function getAuthorizationHeader():
string {
        if (function_exists('getallheaders')) {
            foreach (getallheaders() as $nama => $nilai)
{
                if (strcasecmp($nama, 'Authorization')
=== 0) {
                    return trim($nilai);
                }
            }
        }
        return trim($_SERVER['HTTP_AUTHORIZATION'] ??
$_SERVER['REDIRECT_HTTP_AUTHORIZATION'] ?? '');
    }

    // Middleware: hentikan request dengan 401 bila token
tidak ada / tidak valid
    public static function requireAuth(): array {
        $authHeader = self::getAuthorizationHeader();

        if (!preg_match('/^Bearer\s+(\S+)$/i',
$authHeader, $m)) {
            header('WWW-Authenticate: Bearer');
            Response::json(401, false, 'Token tidak
ditemukan');
        }
```

```
        $payload = self::validateToken($m[1]);
        if (!$payload) {
            header('WWW-Authenticate: Bearer');
            Response::json(401, false, 'Token tidak valid
atau kedaluwarsa');
        }
        return $payload;
    }
}
```

Method `generateToken()` membentuk header dan payload JWT, menambahkan waktu kedaluwarsa, kemudian menghasilkan signature menggunakan algoritma HMAC-SHA256. Token yang terbentuk digunakan sebagai bukti autentikasi saat client mengakses endpoint yang dilindungi.

Selain itu, class `Auth` memiliki method `validateToken()` untuk memeriksa signature dan masa berlaku token. Method `requireAuth()` digunakan untuk membaca Bearer Token dari header request dan mengembalikan respons HTTP 401 apabila token tidak ditemukan atau tidak valid.

B. Langkah 2 — Menambahkan Aksi `login()` pada AuthController

Langkah kedua adalah menambahkan method `login()` pada file `app/controllers/AuthController.php`. Method ini digunakan untuk memverifikasi username dan password pengguna yang telah terdaftar.

Kode implementasinya adalah sebagai berikut:

```
 public function login(): void {
        $input = $this->readJson();

        $username = $this->str($input, 'username');
        $password = isset($input['password']) &&
is_scalar($input['password']) ? (string)
$input['password'] : '';

        $missing = [];
        if ($username === '') $missing['username'] =
'Field username wajib diisi';
```

```
        if ($password === '') $missing['password'] =
'Field password wajib diisi';
        if ($missing) {
            Response::json(400, false, 'Username dan
password wajib diisi', null, $missing);
        }

        $user = $this->model->findByUsername($username);

        // Pesan sama untuk username salah maupun
password salah
        if (!$user || !password_verify($password,
$user['password'])) {
            Response::json(401, false, 'Username atau
password salah');
        }

        $token = Auth::generateToken(['id' => (int)
$user['id'], 'role' => $user['role']]);

        Response::json(200, true, 'Login berhasil', [
            'token'      => $token,
            'role'       => $user['role'],
            'expires_in' => Auth::TOKEN_TTL,
        ]);
    }
}
```

Method `login()` membaca data JSON yang dikirim melalui request. Selanjutnya, sistem mencari pengguna berdasarkan username dan memeriksa password menggunakan `password_verify()`. Apabila username atau password salah, API mengembalikan HTTP 401. Jika kredensial benar, sistem menghasilkan JWT dan mengirimkan token beserta role pengguna melalui respons HTTP 200.

C. Langkah 3 — Mendaftarkan Resource `login` pada Router

Langkah ketiga adalah mendaftarkan resource `login` pada file `Router.php` agar request ke endpoint login diarahkan ke `AuthController`.

Pada bagian `$map`, resource `login` ditambahkan sebagai berikut:

```php
<?php
class Router {

    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController',
        'register'  => 'AuthController',
        'login'     => 'AuthController',
    ];
```

Selanjutnya, di dalam method `dispatch()`, penanganan khusus untuk resource `login` ditambahkan sebelum blok penanganan `register` dan pemrosesan CRUD.

```php
       if ($resource === 'login') {
                if ($_SERVER['REQUEST_METHOD'] !==
'POST') {
                    header('Allow: POST');
                    Response::json(405, false, 'Method
tidak diizinkan, endpoint login hanya menerima POST');
                }
                $controller->login();
                return;
            }

            // register bukan resource CRUD: hanya
menerima POST
            if ($resource === 'register') {
                if ($_SERVER['REQUEST_METHOD'] !==
'POST') {
                    header('Allow: POST');
                    Response::json(405, false, 'Method
tidak diizinkan, endpoint register hanya menerima POST');
                }
                $controller->register();
                return;
            }
```

Dengan penambahan tersebut, request ke endpoint `/login` diarahkan ke method `login()` pada `AuthController`. Implementasi ini mengikuti prosedur BKPM. Jika ingin membatasi endpoint hanya untuk metode POST, pemeriksaan metode HTTP dapat ditambahkan sesuai kebutuhan.

D. Langkah 4 — Melindungi Method Controller dengan `Auth::requireAuth()`

Langkah keempat adalah melindungi operasi penambahan, perubahan, dan penghapusan data mahasiswa menggunakan autentikasi. Pemanggilan `Auth::requireAuth()` ditempatkan pada baris awal method `store()`, `update()`, dan `destroy()` pada file `MahasiswaController.php`.

Contoh penerapannya pada method `store()` adalah sebagai berikut:

```
// POST /mahasiswa
  public function store(): void
  {
      Auth::requireAuth();

      $data = $this->clean($this->readJson());

      try {
          $id = $this->model->create($data);
      } catch (DomainException $e) {
          Response::json(409, false, $e->getMessage());
      }

      Response::json(
          201,
          true,
          'Data berhasil ditambahkan',
          ['id' => (int) $id]
      );
  }

  // PUT /mahasiswa?id=1
  public function update($id = null): void
  {
      Auth::requireAuth();
```

```
        $id = $this->requireId($id);

        if (!$this->model->find($id)) {
            Response::json(404, false, 'Data tidak
ditemukan');
        }

        $data = $this->clean($this->readJson());

        try {
            $this->model->update($id, $data);
        } catch (DomainException $e) {
            Response::json(409, false, $e->getMessage());
        }

        Response::json(200, true, 'Data berhasil
diperbarui');
    }

    // DELETE /mahasiswa?id=1
    public function destroy($id = null): void
    {
        Auth::requireAuth();

        $id = $this->requireId($id);

        if (!$this->model->find($id)) {
            Response::json(404, false, 'Data tidak
ditemukan');
        }

        $this->model->delete($id);

        Response::json(200, true, 'Data berhasil
dihapus');
```

```
    }
```

Pola yang sama diterapkan pada method `update()` dan `destroy()`, sebelum proses perubahan atau penghapusan data dilakukan.

Method `index()` yang digunakan untuk mengambil data mahasiswa tetap dibiarkan publik untuk kebutuhan pengujian sesuai prosedur BKPM. Dengan demikian, operasi yang dilindungi memerlukan token yang valid sebelum diproses oleh controller.

E. Langkah 5 — Menerapkan Try-Catch pada Lapisan Model

Langkah kelima adalah menerapkan mekanisme error handling pada lapisan model. Penanganan kesalahan dilakukan dengan menggunakan `try-catch` untuk menangani masalah yang dapat terjadi saat proses penyimpanan data ke basis data.

Contoh penerapan pada method `create()` adalah sebagai berikut:

```
 public function create(array $data): string {
        if ($this->nimExists($data['nim'])) {
            throw new DomainException('NIM sudah
terdaftar');
        }
        try {
            $stmt = $this->db->prepare(
                'INSERT INTO mahasiswa (nim, nama,
prodi, email) VALUES (:nim, :nama, :prodi, :email)'
            );
            $stmt->execute([
                ':nim'   => $data['nim'],
                ':nama'  => $data['nama'],
                ':prodi' => $data['prodi'],
                ':email' => $data['email'] ?? null,
            ]);
            return $this->db->lastInsertId();
        } catch (PDOException $e) {
            error_log($e->getMessage());
            Response::json(500, false, 'Terjadi
kesalahan pada server');
        }
```

```
    }

    public function update(int $id, array $data):
void {
        if ($this->nimExists($data['nim'], $id)) {
            throw new DomainException('NIM sudah
digunakan mahasiswa lain');
        }
        try {
            $stmt = $this->db->prepare(
                'UPDATE mahasiswa SET nim = :nim,
nama = :nama, prodi = :prodi, email = :email WHERE
id = :id'
            );
            $stmt->execute([
                ':nim'   => $data['nim'],
                ':nama'  => $data['nama'],
                ':prodi' => $data['prodi'],
                ':email' => $data['email'] ?? null,
                ':id'    => $id,
            ]);
        } catch (PDOException $e) {
            error_log($e->getMessage());
            Response::json(500, false, 'Terjadi
kesalahan pada server');
        }
    }

    public function delete(int $id): void {
        try {
            $stmt = $this->db->prepare('DELETE FROM
mahasiswa WHERE id = :id');
            $stmt->execute([':id' => $id]);
        } catch (PDOException $e) {
            error_log($e->getMessage());
```

```
            Response::json(500, false, 'Terjadi
kesalahan pada server');
        }
    }
}
```

Blok `try` digunakan untuk menjalankan query penyimpanan data. Apabila terjadi kesalahan basis data yang menghasilkan `PDOException`, blok `catch` akan menangani kesalahan tersebut dan mengembalikan respons HTTP 500 dalam format JSON.

Potongan kode tersebut merupakan contoh sesuai prosedur BKPM. Penerapannya perlu disesuaikan dengan struktur method `create()` yang telah digunakan pada model mahasiswa di proyek.

F. Langkah 6 — Pengujian Alur Autentikasi

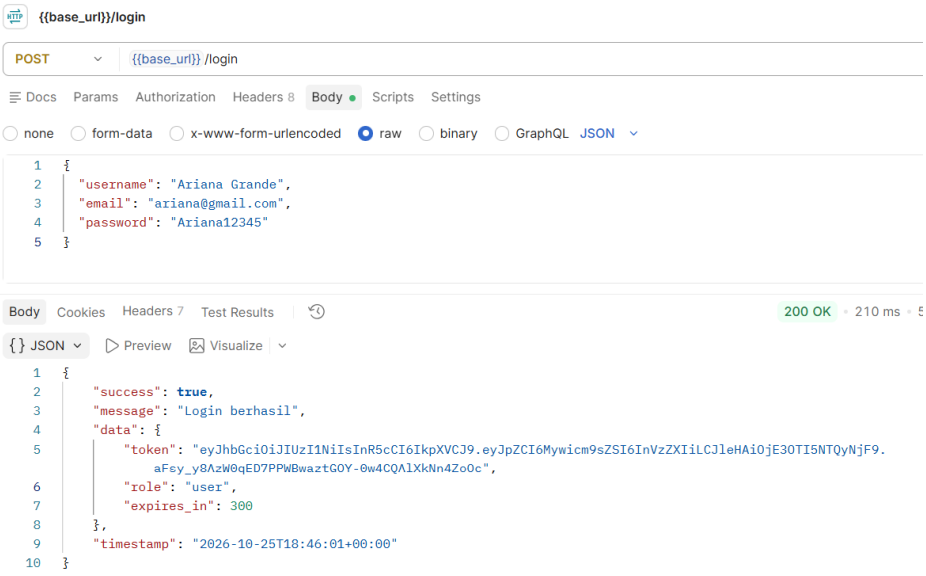
Pengujian alur autentikasi dilakukan menggunakan aplikasi Postman untuk memastikan bahwa proses login dan perlindungan endpoint API berjalan sesuai dengan implementasi yang telah dibuat. Pengujian meliputi proses login menggunakan username dan password, pemeriksaan token autentikasi, serta pengujian akses endpoint mahasiswa menggunakan token.

a. Pengujian Login

Pengujian login dilakukan dengan membuat request baru pada Postman menggunakan konfigurasi berikut:

- **Method:** POST
- **URL:** `{{base_url}}/login`
- **Body:** raw dengan format JSON

Data yang dikirimkan pada request login terdiri atas username dan password pengguna. Contoh format request adalah sebagai berikut:

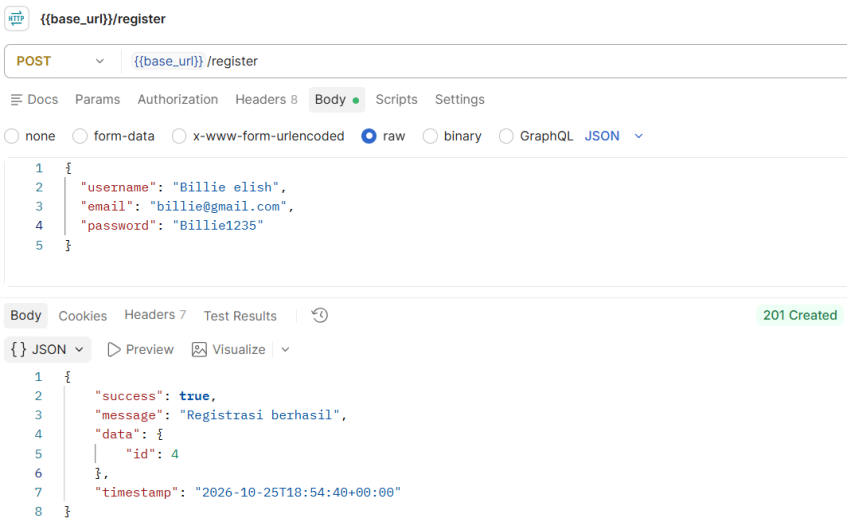


Pada pengujian awal, request menghasilkan respons HTTP `401 Unauthorized` dengan pesan *Username atau password salah*. Hal tersebut terjadi karena akun Ariana Grande belum terdaftar di database sehingga sistem tidak dapat memverifikasi kredensial yang dikirimkan.

b. Registrasi Akun untuk Pengujian Login

Agar pengujian login dapat dilanjutkan, akun pengguna perlu didaftarkan terlebih dahulu melalui endpoint registrasi. Request registrasi menggunakan konfigurasi berikut:

- **Method:** POST
- **URL:** `{{base_url}}/register`
- **Body:** raw dengan format JSON



Setelah request dikirimkan, respons Postman diperiksa untuk memastikan apakah akun berhasil dibuat. Apabila registrasi berhasil, sistem diharapkan memberikan status HTTP `201 Created`. Pengujian login kemudian dilakukan kembali menggunakan username dan password yang sama dengan data registrasi.

c. Pengujian Token Autentikasi

Setelah login berhasil, sistem diharapkan mengembalikan token autentikasi dalam respons JSON. Token tersebut digunakan untuk mengakses endpoint mahasiswa yang dilindungi menggunakan method `Auth::requireAuth()`.

Pengujian dilakukan dengan langkah-langkah berikut:

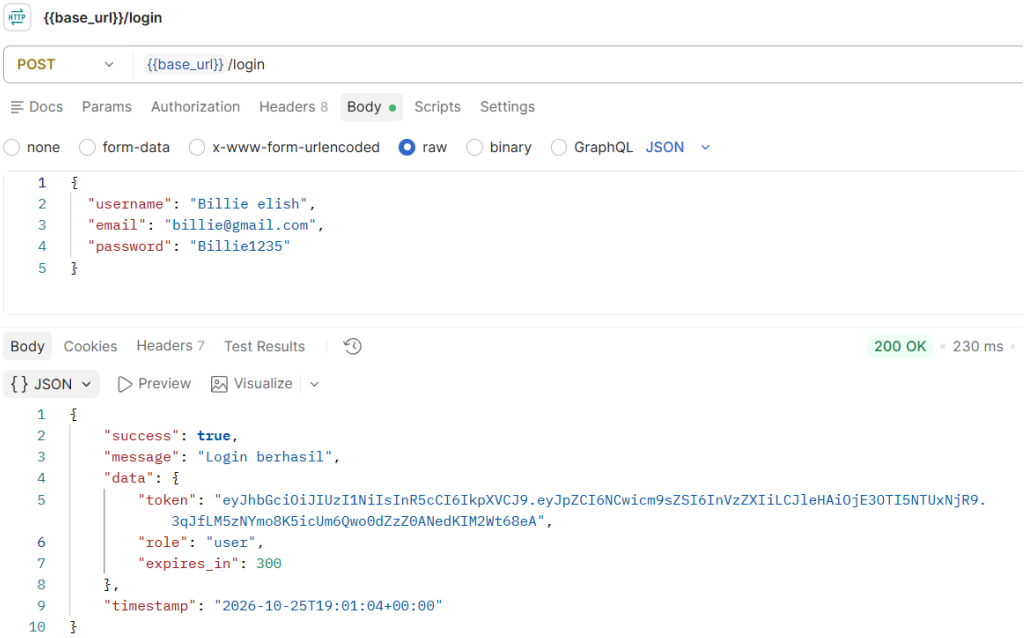
1. Menyalin token dari respons login.

a) Login untuk mendapatkan token

Di Postman, buat request:

- Method: `POST`
- URL: `{{base_url}}/login`
- Body → raw → JSON

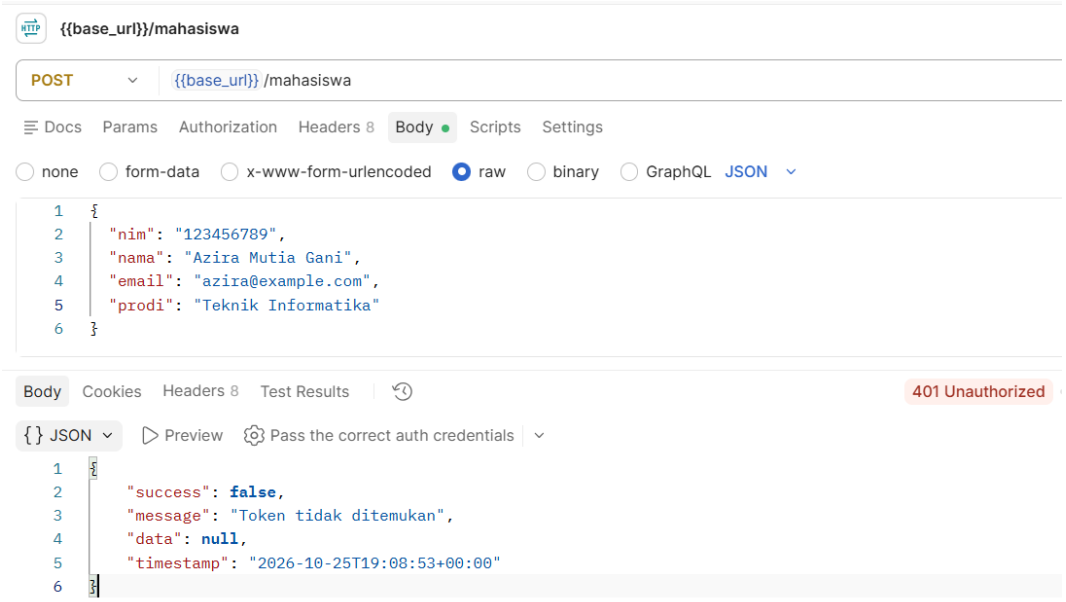
Klik Send. Jika login berhasil, hasil yang diharapkan adalah `200 OK` dengan respons seperti berikut:



2. Uji endpoint tanpa token

POST `{{base_url}}/mahasiswa`

- Buat request baru dengan URL tersebut.
- Buka tab Authorization.
- Pilih No Auth.
- Buka Body → raw → JSON, lalu isi data mahasiswa yang valid sesuai kebutuhan controller.
- Klik Send.

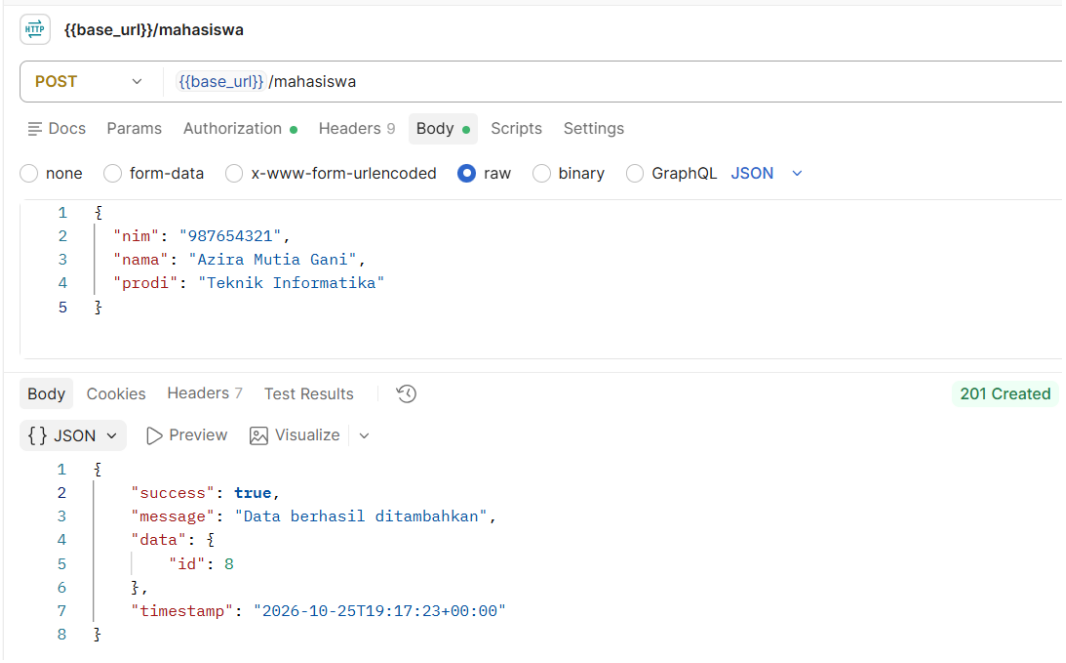


Hasil yang diharapkan adalah `401 Unauthorized`, karena method `store()` menggunakan `Auth::requireAuth()`.

3. Uji endpoint dengan token valid

POST `{{base_url}}/mahasiswa`

- Buka tab Authorization.
- Pilih Bearer Token.
- Tempel token asli dari hasil login.
- Kirim kembali request dengan Body data mahasiswa yang valid.



Hasil yang diharapkan: request melewati pemeriksaan autentikasi dan diproses lebih lanjut. Jika data valid dan belum ada NIM yang sama, hasilnya seharusnya `201 Created`.

4. Uji token yang diubah

Ulangi request `POST {{base_url}}/mahasiswa`, tetapi ubah satu karakter pada token di bagian Bearer Token.

Hasil yang diharapkan adalah `401 Unauthorized` karena token tidak valid.

- Membuka request endpoint mahasiswa yang memerlukan autentikasi.
- Memilih tab **Authorization** pada Postman.
- Memilih tipe autentikasi **Bearer Token**.
- Memasukkan token yang diperoleh dari proses login.
- Mengirimkan request dan memeriksa respons yang diterima.

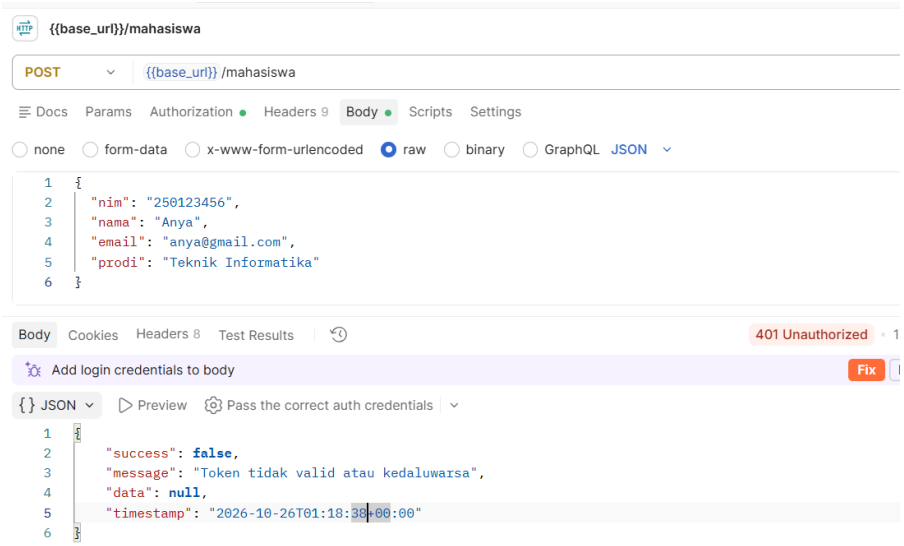
1) Token sebelumnya:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NSwicm9sZSI6InVzZXIiLCJleHAiOjE3OTI5NzcyOTd9.K6ZA4RMhvMKBS6wz5Nvbvlo8QYuqRj4CiWTF5orxajM

2) Token diubah:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NSwicm
9sZSI6InVzZXIiLCJleHAiOjE3OTI5NzcyOTd9.K6ZA4RMhvMKB
S6wz5Nvbvlo8QYuqRj4CiWTF5orxajz

Hasil:



Selain menggunakan token valid, pengujian juga dilakukan tanpa token dan dengan token yang telah diubah untuk memeriksa apakah sistem menolak permintaan yang tidak memiliki autentikasi valid.

D. Hasil Pengujian

Hasil pengujian dicatat berdasarkan respons aktual yang diperoleh dari Postman. Tabel berikut digunakan untuk mendokumentasikan hasil setiap skenario pengujian.

No.	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual
1	Login dengan akun yang belum terdaftar	HTTP 401 Unauthorized	HTTP 401 Unauthorized
2	Registrasi akun baru	HTTP 201 Created	Diisi setelah pengujian
3	Login dengan akun terdaftar dan kredensial benar	HTTP 200 OK dan token diterima	Diisi setelah pengujian

4	Mengakses endpoint tanpa token	HTTP 401 Unauthorized	Diisi setelah pengujian
5	Mengakses endpoint dengan token valid	Permintaan diproses sesuai data yang dikirim	Diisi setelah pengujian
6	Mengakses endpoint menggunakan token yang diubah	HTTP 401 Unauthorized	Diisi setelah pengujian
7	Mengakses endpoint menggunakan token kedaluwarsa	HTTP 401 Unauthorized	Diisi jika pengujian dilakukan

Berdasarkan pengujian awal, endpoint login telah memberikan respons `401 Unauthorized` ketika kredensial yang dikirimkan tidak dapat diverifikasi. Pengujian lanjutan dilakukan setelah akun terdaftar untuk memeriksa penerbitan token dan pembatasan akses endpoint mahasiswa.

3.1.2 Tugas Praktik Mandiri

A. Menganalisis Sifat Stateless pada JWT

Tugas praktik mandiri pertama adalah menganalisis alasan JWT yang bersifat stateless tidak memiliki proses logout di sisi server secara otomatis tanpa mekanisme tambahan.

JWT dapat diverifikasi melalui signature dan klaim yang terdapat pada token tanpa harus menyimpan sesi login di server. Oleh karena itu, ketika pengguna melakukan logout, token yang sudah diterbitkan masih dapat digunakan sampai kedaluwarsa apabila server tidak memiliki mekanisme untuk mencabutnya.

Salah satu solusi yang dapat diterapkan adalah menggunakan token blacklist, yaitu daftar token yang sudah tidak diizinkan untuk digunakan. Dengan mekanisme tersebut, server dapat menolak token tertentu meskipun masa berlakunya belum habis.

Berdasarkan analisis tersebut, sifat stateless JWT memudahkan proses autentikasi, tetapi membutuhkan mekanisme tambahan apabila aplikasi harus mendukung pencabutan token atau logout yang berlaku di sisi server.

B. Memperpendek Masa Berlaku Token Menjadi Lima Menit

Pada tugas pengembangan ini, dilakukan perubahan masa berlaku JSON Web Token (JWT) menjadi lima menit atau 300 detik. Tujuan dari perubahan ini adalah membatasi waktu penggunaan token untuk meningkatkan keamanan autentikasi pada API. Perubahan dilakukan pada file `app/core/Auth.php`.

1. Mengatur Masa Berlaku Token

Pengaturan masa berlaku token dilakukan dengan menetapkan nilai konstanta `TOKEN_TTL` sebesar 300 detik. Potongan kode yang digunakan adalah sebagai berikut.

```
    const TOKEN_TTL = 300;

    private static function base64url(string $data): string {
        return rtrim(strtr(base64_encode($data), '+/', '-_'), '=');
    }
```

Kode tersebut menetapkan masa berlaku token selama 300 detik atau lima menit.

2. Menentukan Waktu Kedaluwarsa Token

Nilai `TOKEN_TTL` digunakan pada fungsi `generateToken()` untuk menentukan waktu kedaluwarsa token. Potongan kode yang digunakan adalah sebagai berikut.

```
        $payload['exp'] = time() + self::TOKEN_TTL;
```

Kode tersebut menambahkan waktu saat token dibuat dengan masa berlaku 300 detik. Hasil perhitungan disimpan pada properti `exp` sebagai informasi waktu kedaluwarsa token.

3. Memeriksa Masa Berlaku Token

Pemeriksaan masa berlaku token dilakukan pada fungsi `validateToken()`. Potongan kode yang digunakan adalah sebagai berikut.

```
    if (!is_array($data) || !isset($data['exp']) ||
$data['exp'] < time()) return false;
```

Kode tersebut memeriksa apakah data token valid, apakah informasi kedaluwarsa tersedia, dan apakah waktu kedaluwarsa telah terlewati. Jika salah satu kondisi tersebut tidak terpenuhi, fungsi akan mengembalikan nilai `false`, sehingga token ditolak oleh sistem autentikasi.

4. Pengujian Menggunakan Postman

Pengujian dilakukan dengan memperoleh token melalui endpoint `POST /login`, kemudian menggunakan token tersebut untuk mengakses endpoint `POST /mahasiswa`. Setelah masa berlaku token berakhir, request dikirim kembali menggunakan token yang sama untuk memeriksa apakah sistem menolak token yang telah kedaluwarsa.

Hasil yang diharapkan dari pengujian ini adalah respons `401 Unauthorized` dengan pesan *"Token tidak valid atau kedaluwarsa"*. Respons tersebut menunjukkan bahwa token tidak lagi dapat digunakan setelah melewati masa berlakunya.

a) Bukti Praktikum

- `Auth.php`

```
<?php
class Auth {
    const SECRET = 'kunci_rahasia_workshop_2026';
    // Tugas Mandiri 2: masa berlaku token 5 menit
(300 detik)
    const TOKEN_TTL = 300;

    private static function base64url(string
$data): string {
```

```
        return rtrim(strtr(base64_encode($data),
'+/', '-_'), '=');
    }

    private static function base64urlDecode(string
$data) {
        $sisa = strlen($data) % 4;
        if ($sisa) {
            $data .= str_repeat('=', 4 - $sisa);
        }
        return base64_decode(strtr($data, '-_',
'+/'), true);
    }

    public static function generateToken(array
$payload): string {
        $header     =
self::base64url(json_encode(['alg' => 'HS256',
'typ' => 'JWT']));
        $payload['exp'] = time() + self::TOKEN_TTL;
        $payloadEnc =
self::base64url(json_encode($payload));
        $signature  =
self::base64url(hash_hmac('sha256',
"$header.$payloadEnc", self::SECRET, true));
        return "$header.$payloadEnc.$signature";
    }

    public static function validateToken(string
$token) {
        $parts = explode('.', $token);
        if (count($parts) !== 3) return false;

        [$header, $payload, $signature] = $parts;
```

```
        $valid =
self::base64url(hash_hmac('sha256',
"$header.$payload", self::SECRET, true));
        if (!hash_equals($valid, $signature))
return false;

        $json = self::base64urlDecode($payload);
        $data = $json === false ? null :
json_decode($json, true);
        if (!is_array($data)
|| !isset($data['exp']) || $data['exp'] < time())
return false;

        return $data;
    }

    // Ambil header Authorization (tidak peka huruf
besar/kecil)
    private static function
getAuthorizationHeader(): string {
        if (function_exists('getallheaders')) {
            foreach (getallheaders() as $nama =>
$nilai) {
                if (strcasecmp($nama,
'Authorization') === 0) {
                    return trim($nilai);
                }
            }
        }
        return
trim($_SERVER['HTTP_AUTHORIZATION'] ??
$_SERVER['REDIRECT_HTTP_AUTHORIZATION'] ?? '');
    }

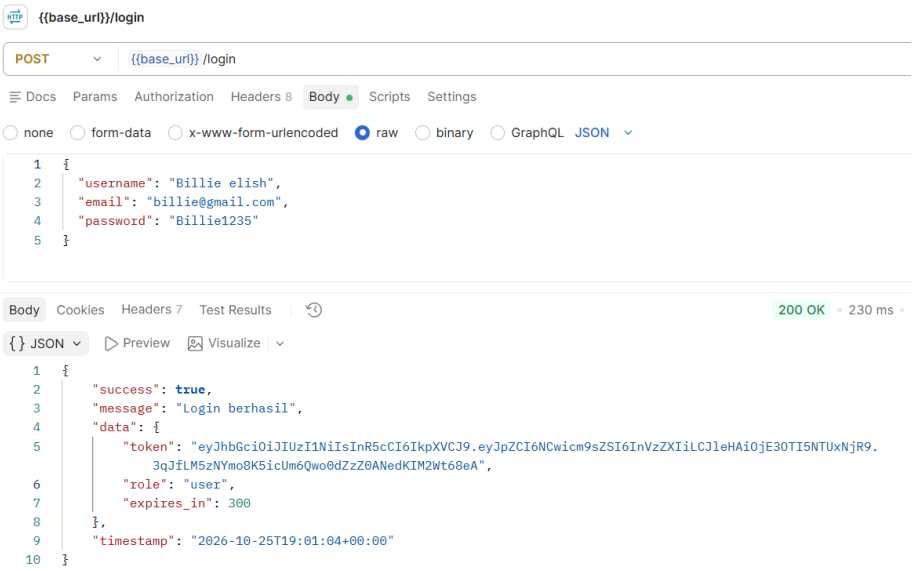
    // Middleware: hentikan request dengan 401 bila
token tidak ada / tidak valid
```

```
    public static function requireAuth(): array {
        $authHeader =
self::getAuthorizationHeader();

        if (!preg_match('/^Bearer\s+(\S+)$/i',
$authHeader, $m)) {
            header('WWW-Authenticate: Bearer');
            Response::json(401, false, 'Token tidak
ditemukan');
        }

        $payload = self::validateToken($m[1]);
        if (!$payload) {
            header('WWW-Authenticate: Bearer');
            Response::json(401, false, 'Token tidak
valid atau kedaluwarsa');
        }
        return $payload;
    }
}
```

- Hasil Dokumentasi



BAB 4. PEMBAHASAN

4.1 Pembahasan

Praktikum Acara 20 membahas penerapan autentikasi menggunakan JSON Web Token (JWT) dan penanganan kesalahan pada RESTful API Sistem Informasi Akademik. Pengembangan dilakukan dengan melanjutkan fitur registrasi pengguna dari praktikum sebelumnya menggunakan PHP Native dan MySQL.

Class `Auth` pada file `app/core/Auth.php` digunakan untuk membuat dan memvalidasi token JWT. Token dibuat setelah pengguna berhasil melakukan login dengan username dan password yang sesuai. Token tersebut berisi informasi pengguna dan waktu kedaluwarsa yang digunakan untuk memeriksa hak akses pada endpoint tertentu.

Controller `AuthController` menangani proses login dengan memeriksa kelengkapan input, mencari pengguna berdasarkan username, dan mencocokkan password menggunakan `password_verify()`. Apabila autentikasi berhasil, server mengembalikan token JWT yang dapat digunakan untuk mengakses endpoint yang memerlukan autentikasi.

Penerapan `Auth::requireAuth()` pada method penambahan, perubahan, dan penghapusan data mahasiswa digunakan untuk membatasi akses terhadap operasi tersebut. Sementara itu, proses pengambilan data mahasiswa tetap dapat diakses tanpa token sesuai implementasi yang dibuat. Dengan demikian, autentikasi diterapkan pada operasi yang membutuhkan perlindungan akses.

Pengujian menggunakan Postman menunjukkan bahwa proses login menghasilkan respons `200 OK` dengan pesan *"Login berhasil"* dan nilai `expires_in` sebesar 300 detik. Hasil tersebut menunjukkan bahwa konfigurasi masa berlaku token telah ditetapkan selama lima menit.

Pengujian penambahan mahasiswa tanpa token menghasilkan respons `401 Unauthorized` dengan pesan *"Token tidak ditemukan"*. Selain itu, pengiriman data dengan NIM yang sudah terdaftar menghasilkan respons `409 Conflict`. Hasil tersebut menunjukkan bahwa sistem memberikan respons berbeda sesuai kondisi permintaan yang diterima.

4.2 Analisis Keamanan dan Penanganan Kesalahan

Keamanan autentikasi diterapkan melalui validasi tanda tangan token menggunakan algoritma HMAC SHA-256 dan pemeriksaan waktu kedaluwarsa. Pada file `Auth.php`, konstanta `TOKEN_TTL` ditetapkan sebesar 300 detik. Token yang telah melewati masa berlaku akan ditolak ketika digunakan untuk mengakses endpoint yang dilindungi.

Penanganan kesalahan dilakukan dengan mengembalikan status HTTP yang sesuai dengan kondisi permintaan. Status `400 Bad Request` digunakan ketika input atau format JSON tidak valid, `401 Unauthorized` digunakan ketika autentikasi gagal, dan `409 Conflict` digunakan ketika terjadi konflik data seperti NIM yang sudah terdaftar. Mekanisme tersebut membantu client memahami penyebab kegagalan permintaan API.

Meskipun demikian, pengujian token yang diubah menghasilkan pesan *"Token tidak valid atau kedaluwarsa"*. Karena pesan tersebut mencakup dua kemungkinan, hasil itu belum cukup untuk memastikan apakah penolakan terjadi akibat perubahan token atau karena token telah kedaluwarsa. Pengujian ulang menggunakan token baru yang diubah dan langsung dikirim diperlukan untuk memastikan hasilnya.

4.3 Evaluasi Hasil Praktikum

Evaluasi dilakukan berdasarkan respons API yang diperoleh melalui Postman. Login berhasil menghasilkan status `200 OK`, permintaan penambahan mahasiswa tanpa token ditolak dengan status `401 Unauthorized`, dan permintaan menggunakan NIM yang sudah terdaftar ditolak dengan status `409 Conflict`.

Hasil tersebut menunjukkan bahwa mekanisme login, pembatasan akses, dan penanganan konflik data telah memberikan respons sesuai kondisi yang diuji. Pengaturan masa berlaku token juga telah diterapkan selama lima menit melalui konstanta `TOKEN_TTL`.

Untuk memastikan seluruh kebutuhan praktikum terpenuhi, pengujian token kedaluwarsa perlu dilakukan dengan menunggu lebih dari lima menit sebelum mengirim ulang request menggunakan token yang sama. Hasil pengujian tersebut perlu dicatat berdasarkan respons aktual dari server agar kesimpulan laporan sesuai dengan pelaksanaan praktikum.

BAB 5. PENUTUP

5.1 Kesimpulan

Praktikum Acara 20 membahas penerapan API Authentication dan Error Handling pada aplikasi Sistem Informasi Akademik berbasis PHP. Autentikasi dilakukan melalui proses login yang memverifikasi username dan password, kemudian menghasilkan JWT sebagai bukti autentikasi.

Token diperiksa melalui class `Auth` sebelum operasi tertentu pada endpoint mahasiswa dijalankan. Implementasi ini membantu membatasi akses terhadap operasi penambahan, perubahan, dan penghapusan data.

Penanganan kesalahan diterapkan menggunakan `try-catch` untuk menangani masalah pada operasi basis data dan mengembalikan respons JSON dengan status HTTP yang sesuai.

Selain itu, sifat stateless JWT menunjukkan bahwa logout tidak otomatis membatalkan token yang telah diterbitkan. Oleh karena itu, masa berlaku token dan mekanisme pencabutan perlu dipertimbangkan sesuai kebutuhan keamanan aplikasi.

5.2 Saran

Pengembangan berikutnya dapat dilakukan dengan menyimpan kunci rahasia JWT pada konfigurasi lingkungan, memperketat validasi token, serta menerapkan HTTPS pada lingkungan produksi. Pengujian juga perlu mencakup token tidak valid, token kedaluwarsa, kredensial salah, dan kesalahan basis data.